Keras for supervised learning: An introduction
Neural Systems A.Y. 2023-2024

Davide Borra, PhD

November 29, 2023

Contents

1 Introduction **1**

2 Data pre-processing **3**

3 Network design **4**
3.1 Layers for designing fully-connected neural networks 6
3.2 Layers for designing convolutional neural networks 6

4 Network training **6**

5 Network evaluation and feature investigation **9**

1 Introduction

In a classification task it is asked to approximate the relationship $f(X)$ that maps an input (e.g., a grey-scale image containing a handwritten digit from 0 to 9) to the proper label (e.g., the class "8" associated to the input grey-scale image) from a labelled dataset D consisting of M labelled patterns, see Figure 1 and Equation 1. A neural network can be trained in a supervised way to learn to approximate $f(X)$ by learning the function $g(X;\theta)$, parametrized in the array θ that contains all trainable parameters (i.e., the set of all weights and biases of the network), see Equation 2. Please note that in this lecture, to be consistent with the supervised tasks addressed in the exercises of the laboratory session (exercises 11 and 12), it is used the classification of 10 different handwritten digits as reference classification task (see Figure 1).

$$D = \{\left(X^{(0)}, y^{(0)}\right), ..., \left(X^{(p)}, y^{(p)}\right), ... \left(X^{(M-1)}, y^{(M-1)}\right)\},\ 0 \leq p \leq M-1, \tag{1}$$

where the p-th pair (or tuple) $\left(X^{(p)}, y^{(p)}\right)$ contains the input and the desired output (known by the supervisor).

$$g(X;\theta) : \mathbb{R}^{N \times N} \longrightarrow L = \{l_0, ..., l_{C-1}\}, \tag{2}$$

where N denotes the row (=column) dimension of the input 2-D square matrix, C represents the number of overall classes (e.g., $C = 10$ when classifying 10 digits, from 0 to 9), and $L = \{l_0, ..., l_{C-1}\}$ is the set of the possible labels.

Specifically, the network accepts as input the grey-scale image X, and provides as output the probability that the input image belongs to each class (i.e., softmax-activated neurons), $p(l_k|X;\theta), 0 \leq k \leq C-1$.

As very first step, the network inputs and labels should be pre-processed and split into separate training and test sets (*data pre-processing and split*, step 0). The pre-processing stage of the inputs usually consists of simple steps, e.g., downsampling the image to match the desired dimension and scaling the image (e.g., to $[0, 1]$ range) or standardizing the image (e.g., to zero mean and unitary

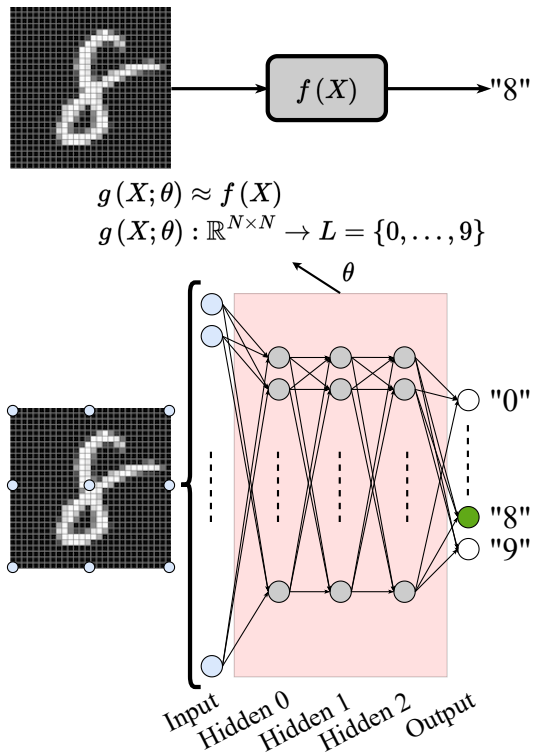

Figure 1: Handwritten digit classification (10 classes). A neural network (e.g., a "deep" fully-connected network with 3 hidden layer in the figure) can be trained to approximate the function $f(X)$, mapping an input grey-scale image containing a handwritten digit to the proper class (e.g., "8"). Specifically, the network accepts as input the grey-scale image, and provides as output the probability that the input image belongs to each class (i.e., softmax-activated neurons), $p(l_k|X;\theta), 0 \leq k \leq C-1$. The learned function $g(X;\theta)$ is parametrized in θ, containing all network weights and biases. Here, N denotes the row (=column) dimension of the input 2-D square matrix.

standard deviation) to facilitate the network training. However, when using noisy inputs, e.g., electroencephalographic signals arranged as 2-D inputs (with electrode sites along rows and time steps along columns), the pre-processing stage is also devoted to clean the input representation from artifacts (e.g., blinking, eye movements, muscle activity, etc.), thus producing an input highly representative of the desired activity (e.g., the brain activity in case of electroencephalographic signals).

Then, to train and evaluate a neural network in a supervised task, the network need to be designed in its main layers (*network design*, step 1) and trained using a training set of labelled examples (*network training*, step 2). Lastly, the trained network should be evaluated on a separate test set of labelled examples and could also be investigated in the learned features (*network evaluation and feature investigation*, step 3). These main steps will be addressed in the following sections, separately.

Keras is a Python library devoted to design, train and evaluate deep neural networks from scratch for both supervised (e.g., classification, regression) and unsupervised tasks. This library run on top of the machine learning package TensorFlow (developed by Google), an end-to-end open-source machine learning platform, and it is focused on enabling fast experimentation. Thus, Keras represents a high-level version of TensorFlow, providing "an approachable, highly-productive interface for solving machine learning problems, with a focus on modern deep learning" (from Keras web-site).

Keras comes packaged with TensorFlow 2 (i.e., you can access it as a submodule of TensorFlow: `from tensorflow import keras`) and to start using Keras you should simply install TensorFlow 2, e.g., for the cpu-version (recommended if you do not have a laptop / personal computer with a dedicated graphic processing unit): `pip3 install tensorflow-cpu` or simply install `tensorflow-cpu` from Anaconda navigator.

The aim of this lecture is to provide an introduction to Keras applied to supervised tasks, specifically classification tasks. However, it is worth remarking that this guide can be also exploited to solve other supervised tasks (e.g., a regression task) or unsupervised tasks (e.g., use of an autoencoder), with only few changes. This lecture is paired with the script `keras-intro-supervised.py`.

In the following, the main Keras functionalities needed for the Course will be presented. However, as a general rule when dealing with a new software / software package, it is also strongly recommended to read the associated documentation, for a more in-depth understanding.

2 Data pre-processing

The data pre-processing stage is considered one of the main crucial stages when approaching a supervised problem. Indeed, the most common issues may arise from a wrong input pre-processing and/or a wrong label handling (e.g., errors in the training-test splits). With reference to grey-scale image classification (e.g., handwritten digit classification), the following steps should be performed:

- Split the dataset into separate train and test partitions. Large datasets could be split into a fixed train-test partition to evaluate machine learning algorithms (e.g., as with ImageNet, one of the largest datasets used to train deep neural networks on RGB images, consisting of $> 1M$ examples). However, generally (and especially in biomedical applications) datasets are small (i.e., few hundred of examples). Therefore, in this case, K-fold cross-validation should be used.

- Resampling (e.g., downsampling) the input 2-D matrix to the desired input dimension.

- Scaling or standardizing the input 2-D matrix. Each image could be scaled to have values in the $[0, 1]$ range, or to have zero mean (across rows, columns and examples) and unitary standard deviation (across rows, columns and examples).

- Inserting an additional singleton dimension to the input 2-D matrix (in case of grey-scale images). When handling grey-scale images, the matrix representing the image is a 2-D matrix; however, when handling colored images (e.g., RGB images), the matrix representing the image is a 3-D matrix, where the third dimension represents the "channel" dimension (red, blue, and green channels). Therefore, Keras (but also other Python packages operating in general on multi-channel images), when dealing with a 2-D input, expects the input to have also a third dimension corresponding to the "channel" dimension. Of course, in case of a grey-scale image, this corresponds to simply replicating the 2-D matrix in a slice of a 3-D matrix (i.e., using a single input feature map), e.g., for a 2-D input matrix of 28×28 Keras expects a 3-D input matrix of $28 \times 28 \times 1$.

- Convert labels using "label encoding" or to "one-hot encoding". Generally, labels are provided as categorical values, e.g., {"cat", "dog", "bird"} for cat vs. dog vs. bird classification or {"0", ..."9"} for handwritten digit classification. Therefore, to properly compute the error in the network prediction, i.e., the loss between the network output and the ground truth label provided by the supervisor, it is needed to convert labels. With a label encoding, the label of each example is transformed by associating each class to a progressive number from 0 to $C-1$, e.g., {0: "cat", 1:"dog", 2: "bird"}. With one-hot encoding, the label of each example is represented by an array of C values, all at 0 except for the element at the index corresponding to the label code (from label encoding), which is set to 1. That is, from the previous example, for the label "dog", once applied label encoding the label become 1. Then, once applied one-hot encoding the label is [0, 1, 0] (all values are at 0 except for the column corresponding to the label code, i.e., the second column).

When addressing a handwritten digit classification task, the most used dataset is called "MNIST" (publicly available here) and is composed by 60000 training and 10000 test grey-scale images with size 28x28. Keras offers data loading methods to easily download and load in memory the most common benchmark datasets, e.g., for MNIST via `keras.datasets.mnist.load_data` (see here for a list of the benchmark datasets included in Keras). See Listing 1 for a snippet code for downloading, loading and preparing the MNIST dataset.

```
N = 28 # number of rows and columns of the input square matrix
C = 10 # number of total classes (e.g., 10 possible digits)

# load the data and split it between train and test sets
(x_train, labels_train), (x_test, labels_test) = keras.datasets.mnist.load_data() #
    uint8 (0-255)
print("Input data type:", x_train.dtype)
print("Min value:", x_train.min())
print("Max value:", x_train.max())
print("Shape of the training examples:", x_train.shape)
print("Shape of the test examples:", x_test.shape)

# scale images to the [0, 1] range
x_train = x_train / 255
x_test = x_test / 255
print("Input data type (after scaling):", x_train.dtype)
print("Min value (after scaling):", x_train.min())
print("Max value (after scaling):", x_train.max())

# make sure images have shape (28, 28, 1)
n_examples_train = x_train.shape[0]
n_examples_test = x_test.shape[0]
x_train = x_train.reshape((n_examples_train, N, N, 1))
x_test = x_test.reshape((n_examples_test, N, N, 1))

print("Shape of the training examples (after reshaping):", x_train.shape)
print("Shape of the test examples (after reshaping):", x_test.shape)

# convert labels to one-hot encoded labels using a custom method (to be defined in the
    exercises 11 and 12)
print("Shape of the trainin labels:", labels_train.shape)
print("Shape of the test labels:", labels_test.shape)
y_train = to_categorical(labels_train, C)
y_test = to_categorical(labels_test, C)
print("Shape of the trainin labels (after one-hot encoding):", y_train.shape)
print("Shape of the test labels (after one-hot encoding):", y_test.shape)
```

Listing 1: Data preparation (example on the MNIST dataset).

3 Network design

To design a network consisting of a stack of ordered layers of interconnected neurons, Keras offers a useful `keras.Sequential` class, converting a list of layers into a model, automatically creating all the needed connections across units. In turn, each layer should be defined using Keras built-in layers

contained in `keras.layers` submodule. As an example, by defining: `keras.layers.Dense(10)` a 1-D layer of 10 neurons fully-connected with the preceding layer is defined (i.e., a "densely" connected layer, or "dense" layer). It is worth remarking that in this lecture (and in the exercises 11 and 12 of the laboratory session), the focus will be on feed-forward neural networks (not recurrent neural networks), specifically on fully-connected and convolutional neural networks. However, the presented concepts can be exploited also when designing networks with feedback connections. When designing the model, there is no need to explicitly define how to perform the forward-pass (and backward-pass) of the information at each layer, as Keras does this for you, providing an easy way to build and initialize the desired deep neural network. In addition, once defined the network, all weights are automatically randomly initialized. Lastly, the details about the network structure (e.g., sequence of layers, output shape of hidden layers, number of weights and biases introduced by each layer) can be accessed using the `model.summary` method.

See Listing 2 for a snippet code resuming these concepts while defining a 1-D fully-connected neural network consisting of 3 hidden layers of neurons.

```
input_shape = (N, N, 1) # input dimension (grey-scale image 28 x 28, represented as a single input feature map 28x28x1)

# model design
model = keras.Sequential(
    [
        keras.Input(shape=input_shape), # input layer
        keras.layers.Flatten(), # flatten layer (from multiple dimensions to a 1-D)

        keras.layers.Dense(64, activation="relu"), # 1-D layer of 64 neurons, fully-connected with the preceding layer, activated via rectified linear function (ReLUs)
        keras.layers.Dropout(0.25), # application of dropout to the output of the preceding layer with a dropout probability of 0.25 (25% neurons dropped out during training)

        keras.layers.Dense(128, activation="relu"), # 1-D layer of 128 neurons, fully-connected with the preceding layer, activated via rectified linear function (ReLUs)
        keras.layers.Dropout(0.25), # application of dropout to the output of the preceding layer with a dropout probability of 0.25 (25% neurons dropped out during training)

        keras.layers.Dense(128, activation="relu"), # 1-D layer of 128 neurons, fully-connected with the preceding layer, activated via rectified linear function (ReLUs)
        keras.layers.Dropout(0.25), # application of dropout to the output of the preceding layer with a dropout probability of 0.25 (25% neurons dropped out during training)

        keras.layers.Dense(C, activation="softmax"), # output layer with C neurons, fully-connected with the preceding layer, activated via softmax
    ]
)

# inspect model
model.summary()
```

Listing 2: Definition of a network (fully-connected network) as a "sequential" list of layers.

The main methods / classes needed to build and inspect a network are resumed in the following.

- `keras.Sequential`. Class that converts a list of layers into a model, creating all connections across neurons between layers. The list of layers has to be provided as input parameter.
- `model.summary`. Method that prints details about the network (e.g., layer names, output shapes, number of trainable parameters).

Depending on the desired network structure, different Keras built-in layers can be used. In the following subsections, the main layers to design fully-connected and convolutional neural networks are presented and briefly discussed. Of course, to deeply understand each of these layers, it is also recommended to read the Keras documentation for defining layers.

3.1 Layers for designing fully-connected neural networks

In the following are listed and briefly discussed the main layers useful for designing a fully-connected network. See also the snippet code reported in Listing 2.

- `keras.Input`. Layer that simply replicates the input provided to the network. You need to specify the input shape as input parameter.
- `keras.layers.Flatten`. Layer that transforms an input multi-dimensional array (e.g., the input matrix $28 \times 28 \times 1$) to a 1-D array (e.g., a 1-D array with 784 elements). It is equivalent to a multi-dimension to 1-D reshaping function. It could be useful to transform input feature maps (e.g., the input image) or hidden feature maps (e.g., as obtained by a convolutional layer) to one 1-D array, that could be then used as input for a following 1-D fully-connected layer.
- `keras.layers.Dense`. Layer of neurons organized in 1-D, fully-connected with the neurons belonging to the preceding layer. The number of units must be specified as input. In addition, you can also specify the activation function to apply to each neuron, e.g., via rectified linear functions.
- `keras.layers.Activation`. Layer that apply an activation function element-wise to the input. The function must be defined as input parameter (e.g., `keras.layers.Activation(‘‘relu’’)` for a rectified linear unit). If neurons were already activated in the `keras.layers.Dense` or `keras.layers.Conv2D` layers, you should not add this layer.
- `keras.layers.Dropout`. Layer that apply dropout to the input at a specific dropout probability. The dropout probability must be provided as input. Dropout will be applied during training (neurons dropped out during training), and will not be performed during network evaluation (all neurons considered).

3.2 Layers for designing convolutional neural networks

In addition to the previous layers, the following layers are useful for designing a convolutional neural network.

- `keras.layers.Conv2D`. Layer of neurons organized in 3-D, where each slice corresponds to the result of the 2-D convolution with a specific convolutional kernel over the output of the preceding layer. Convolutional kernels are learned during training. The number of kernels and the kernel size need to be specified as input parameters. In addition, you can also specify the activation function to apply to each neuron, e.g., via rectified linear functions.
- `keras.layers.MaxPool2D`. Layer that performs max pooling in 2-D on the input. The pooling size need to be specified as input parameter.

4 Network training

Once designed the network, the network parameters (all contained in the parameter array θ) need to be optimized by using a gradient descent-based algorithm (e.g., stochastic gradient descent with momentum) and applying backpropagation to compute gradients. At first, the model needs to be configured (or "compiled") for training, by specifying the desired loss function to minimize (e.g., cross-entropy in case of classification), the specific gradient descend-based algorithm (called "optimizer") to use, and (eventually) other metrics to track during the optimization process (e.g., the accuracy) in addition to the loss. All these training options have to be specified by calling the `model.compile` method (where `model` is the variable containing the previously defined model). Then, the optimization process can start by calling the `model.fit` method, providing as input, in addition to the number of training epochs, and the training inputs and labels, also the mini-batch size, validation data and "callbacks". In the following, these novel input parameters are briefly described.

The mini-batch size corresponds to the number of training examples that are used to estimate gradients during training ("stochastic" gradient descent).

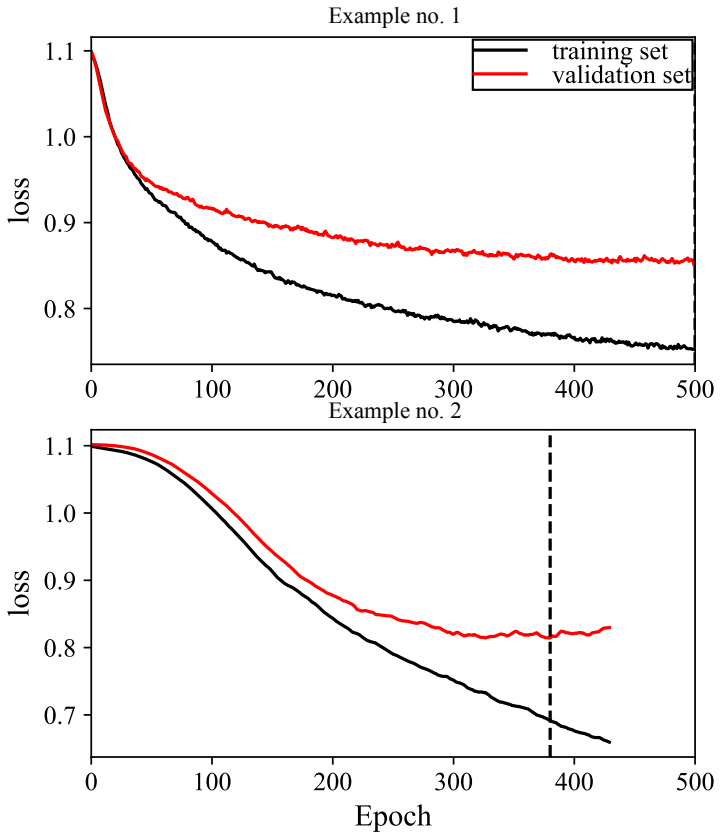

Figure 2: Examples of training and validation losses (cross-entropy) as the training proceed.

The validation data is the pair image + labels belonging to the validation set. In general, a validation set should be extracted from the overall training set (a set of example separate from the training and test sets) to tune the so-called "hyper-parameters" of the neural network, which are non-trainable parameters that define the specific functional form of the network, such as the number of units in the hidden layers, the number of hidden layers, etc. The networks defined using different hyper-parameter values (e.g., using $N_h = 128$ vs. $N_h = 256$ hidden units) are separately trained on the training set and evaluated on the validation set, and the optimal hyper-parameter value is the one associated to the best performance on the validation set (e.g., lowest validation loss or highest validation accuracy). Usually, a portion corresponding to the 10-20% of the overall training set is extracted and used as validation set, while the remaining 80-90% is used to train the network. As an example, let us consider the number of training epochs. This parameter can be considered a hyper-parameter. What is the optimal number of training epochs that enables a better generalization? It is possible to perform a "simple" hyper-parameter tuning on the hyper-parameter "number of training epochs", by training the network for a long training time (enough to guarantee network convergence), evaluating the network on the validation set at each training epoch (i.e., each different hyper-parameter value) and selecting the optimal number of training epochs as the number of epochs corresponding to the lowest validation loss. It is worth remarking that this corresponds to performing early stopping. In Figure 2 are reported two examples of training and validation losses tracked for a long training time (up to 500 training epochs). In the first example the validation loss keep decreasing and no intermediate local minimum (before the last reached epoch) can be observed; conversely, in the second example, the validation loss reached an intermediate local minimum approx. at the epoch 380 (see vertical dashed line in Figure 2). Thus, for this last example, the model trained with approx. 380 epochs represents the best trained model, as evaluated on the separate validation set. This model (trained for 380 epochs) will be then tested on the test set.

As explained above, the Keras functionalities enable to optimize the network parameters by calling a single line of code (i.e., `model.fit` method). Inside this method, the training loop (outer loop over training epochs and inner loop over mini-batches of training data) and backpropagation are performed.

However, with this high-level abstraction, there is no direct access to single epochs or single training iterations (iterations on single mini-batches of data) and thus, it appears not possible to perform evaluations / manipulations on single epochs or training iterations. To solve this apparent limitation, Keras offers functionalities to write custom training loops, implementing training loops from scratch. Furthermore, it is possible to force Keras to call special functions periodically during training (e.g., after each epoch or after each training iteration). These functions are called "callbacks". Among Keras callbacks, the callback `keras.callbacks.ModelCheckpoint` is useful to save only the best model based on a specific metric (e.g., validation loss) during training, thus enabling to store the best model on the validation set (i.e., its weights and biases).

See Listing 3 for a snippet code resuming these concepts while training the network previously defined in Listing 2.

```
lr = 0.001 # learning rate
momentum = 0.9 # momentum term
# defining the optimizer (stochastic gradient descent with momentum)
optimizer = keras.optimizers.SGD(learning_rate=lr, momentum=momentum) # SGD with momentum

# compiling the model=defining the desired loss function to be minimized, the algorithm to use for the optimization process, and other metrics to track
model.compile(loss="categorical_crossentropy", optimizer=optimizer, metrics=["accuracy"])

valid_ratio = 0.1 # ratio of the overall training set to held back as validation set
# extracting the validation set as the first 10% of examples
x_valid = x_train[:round(valid_ratio*n_examples_train),:,:,:]
y_valid = y_train[:round(valid_ratio*n_examples_train),:]
labels_valid = labels_train[:round(valid_ratio*n_examples_train)]
# assigning back the training set as the remaining 90% of examples
x_train = x_train[round(valid_ratio*n_examples_train):,:,:,:]
y_train = y_train[round(valid_ratio*n_examples_train):,:]
labels_train = labels_train[round(valid_ratio*n_examples_train):]

# defining the ModelCheckpoint callback
model_checkpoint_callback = keras.callbacks.ModelCheckpoint(
    filepath='best_mdl.hdf5', # set to 'best_mdl.hdf5' if you want only the best model, overall or to '{epoch:02d}-{val_loss:.5f}.hdf5' if you want to save the best model as the training proceed (best model over time)
    monitor='val_loss', # set to the metric that you want to track for the early stopped model (evaluated offline)
    save_best_only=True)

# start optimizing the network
batch_size = 128 # mini-batch size
max_epochs = 50 # maximum number of epochs
history = model.fit(x_train, y_train,
                    batch_size=batch_size,
                    epochs=max_epochs,
                    validation_data=(x_valid, y_valid),
                    callbacks=[model_checkpoint_callback])

# extracting training and validation losses, training and validation accuracies
train_loss = history.history['loss']
train_acc = history.history['accuracy']
valid_loss = history.history['val_loss']
valid_acc = history.history['val_accuracy']
```

Listing 3: Network training.

The main methods / classes needed to train the network are resumed in the following. Further details about model training can be accessed from the related Keras documentation.

- `keras.optimizers.SGD`. Algorithm that performs stochastic gradient descent with momentum. The learning rate and the momentum term should be specified as input (otherwise, default values will be used).
- `model.compile`. Method that prepare the network for training. Specifically, the desired loss function to minimize, the gradient descent-based algorithm (set from the point above), and other metrics to track during network training should be specified.

- `keras.callbacks.ModelCheckpoint`. Method that tracks a metric (e.g., the validation loss) at each training epoch and save, epoch after epoch, the best model on that metric (models are generally saved with "hdf5" format). The filename and the metric to be monitored have to be specified. Furthermore, if only the best model should be saved, it should be specified as well (see the snippet above). If the saved model is overwritten after each best epoch (it depends on the specified filename), the saved model corresponds to the best model over the entire training (i.e., up to the max number of epochs). This corresponds to the early stopped model.

- `model.fit`. Method that performs the network training. The training set (inputs and labels), number of training epochs, mini-batch size, validation data, and callbacks should be specified as input parameters.

Here are provided some useful online services to further understand what kind of features are extracted from a fully-connected neural network and convolutional neural network, for a digit classification task Furthermore, see here for an example of a convolutional neural network classifying animals/objects contained in RGB images. In the previous links, feed-forward neural networks were trained to classify handwritten digits or different animals/objects in RGB images, and released online to perform inference on new digits (that you can draw) or on new RGB images (that you can upload). As an example, you can try drawing a "0", "1", and "6". Try perturbing the digit containing the "6" by progressively forcing it to be closer to "8". What happens to the softmax-activated output neurons?

5 Network evaluation and feature investigation

Lastly, once trained the network, it should be evaluated on the test set. To do so, the best model, as evaluated on the validation set, should be loaded in memory using the `keras.models.load_model` method. Then, the images belonging to the test set can be provided as input to the network to return the predicted probabilities, i.e., the output of the softmax layer of the network (10 probability values for each test example). To do so, the `model.predict` method can be used (where `model` represents the variable containing the loaded trained model), performing a forward-pass through the network.

Then, the predicted labels can be computed (as the class with highest probability, for each test example), and the confusion matrix can be computed by comparing the predicted labels with the ground truth labels. To this aim, scikit-learn function `sklearn.metrics.confusion_matrix` can be used. In addition, performance measures can be extracted from the confusion matrix. An example of performance measure is the accuracy, defined as the ratio between the number of correctly predicted examples and the number of total examples. However, it is worth remarking that the chosen performance measure strictly depends on the specific supervised task (e.g., classification vs. regression), and on the label distribution. Indeed, in case of highly unbalanced datasets, the accuracy is biased towards the most represented class and different measures should be used (e.g., F1 score). As the MNIST dataset is balanced, in case of handwritten digit classification using MNIST , the accuracy could be used as performance measure.

See Listing 4 for a snippet code that evaluate the performance of the network trained in Listing 3.

```
model = keras.models.load_model('best_mdl.hdf5')
proba_test = model.predict(x_test)
```

Listing 4: Network evaluation.

Lastly, features of trained models can be accessed and visualized / processed, by accessing the `model.layers` attribute of the model class. This is a list containing the layers with all the trained weights and biases. Thus, once identified a target layer to investigate, its learned weights and biases can be accessed with `layer.get_weights()` (where `layer` is the variable associated to the target layer to investigate).

See Listing 5 for a snippet code to investigate the learned weights and biases.

```
layers = model.layers # list containing all trained layers
layer = layers[1] # first fully-connected layer
weights, biases = layer.get_weights() # accessing weights and biases
print(layer.name, weights.shape)
```

```
print(layer.name, biases.shape)
```

Listing 5: Accessing the learned parameters.

The main methods needed to evaluate the network / investigate the learned parameters are resumed in the following. Further details about model evaluation can be accessed from the related Keras documentation.

- `keras.models.load_model`. Method that loads in memory a saved model (e.g., with "hdf5" format). The filepath has to be specified as input parameter.
- `model.predict`. Method of the `model` class that performs a forward-pass through the model. The network input has to be specified as input parameter.
- `model.layers`. Attribute of the `model` class consisting of the list of trained layers.
- `layer.get_weights`. Method of the `layer` class that returns the learned weights and biases as numpy arrays.